

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO
PAULO**

THOMAS BATISTA PINTO

SISTEMA DE BANCO DE DADOS PARA E-COMMERCE COM MONGODB

**CAMPOS DO JORDÃO
2026**

RESUMO

O presente trabalho aborda o desenvolvimento e a implementação de um banco de dados não relacional para um sistema de e-commerce, utilizando o MongoDB como sistema gerenciador e o modelo de dados orientado a documentos. O projeto foi desenvolvido com o objetivo de aprofundar os conhecimentos adquiridos na disciplina de Banco de Dados 2, aplicando ao paradigma NoSQL o sistema de e-commerce anteriormente modelado em banco de dados relacional com MySQL. A modelagem foi realizada sob uma abordagem híbrida: adotou-se o embedding de subdocumentos nos casos em que os dados são consultados em conjunto com maior frequência e utilizaram-se referências entre coleções nos casos em que a reutilização e a consistência dos dados são mais relevantes. O sistema desenvolvido permite o gerenciamento de clientes, produtos, categorias, fornecedores, estoque, pedidos e pagamentos por meio de coleções MongoDB, com dados fictícios inseridos para validação das consultas. No total, trinta consultas foram desenvolvidas utilizando a MongoDB Query Language, abrangendo desde filtros simples até pipelines de agregação com múltiplos estágios, operações de junção entre coleções com \$lookup e subconsultas com \$match e \$expr. Os resultados alcançados demonstram que o modelo de documentos traz benefícios consideráveis em termos de flexibilidade de esquema e desempenho em leitura, ao mesmo tempo em que exige maior cuidado no planejamento da estrutura dos documentos para evitar redundância excessiva e inconsistências. O trabalho contribuiu para o aprofundamento dos conhecimentos em modelagem NoSQL, estratégias de desnormalização e aplicação prática da linguagem MQL em um cenário de e-commerce.

Palavras-chave: Banco de Dados; NoSQL; MongoDB; E-commerce; Modelagem de Documentos.

ABSTRACT

This paper presents the development and implementation of a non-relational database for an e-commerce system, using the MongoDB database management system and the document-oriented data model. The project was developed with the objective of deepening the knowledge acquired in the Database 2 course, transposing into the NoSQL paradigm the e-commerce system previously modeled as a relational database with MySQL. The modeling was conducted under a hybrid approach, combining subdocument embedding in cases where data is frequently queried together, and references between collections in cases where data reuse and consistency are more relevant. The developed system enables the management of customers, products, categories, suppliers, inventory, orders, and payments through MongoDB collections, with fictitious data inserted for query validation. Thirty queries were developed using the MongoDB Query Language, ranging from simple filters to multi-stage aggregation pipelines, cross-collection join operations with \$lookup, and subqueries using \$match and \$expr. The results demonstrate that the document model offers relevant advantages in terms of schema flexibility and read performance, while requiring greater care in planning document structure to avoid excessive redundancy and inconsistencies. The project contributed to the consolidation of knowledge on NoSQL modeling, denormalization strategies, and use of MQL in practical e-commerce contexts.

Keywords: Database; NoSQL; MongoDB; E-commerce; Document Modeling.

LISTA DE ILUSTRAÇÕES

FIGURA 1 – Resultado da Consulta 01 – Clientes cadastrados (O AUTOR, 2026)	27
FIGURA 2 – Resultado da Consulta 02 – Produtos acima de R\$ 500 (O AUTOR, 2026)	27
FIGURA 3 – Resultado da Consulta 03 – Pedidos com status Pago (O AUTOR, 2026)	28
FIGURA 4 – Resultado da Consulta 04 – Estoque abaixo de 20 unidades (O AUTOR, 2026)	28
FIGURA 5 – Resultado da Consulta 05 – Clientes cadastrados em janeiro de 2026 (O AUTOR, 2026)	29
FIGURA 6 – Resultado da Consulta 06 – Pagamento aprovado via Cartão de Crédito (O AUTOR, 2026)	30
FIGURA 7 – Resultado da Consulta 07 – Pedidos pendentes ou cancelados (O AUTOR, 2026)	30
FIGURA 8 – Resultado da Consulta 08 – Produto mais caro (O AUTOR, 2026)	31
FIGURA 9 – Resultado da Consulta 09 – Produto mais barato (O AUTOR, 2026)	31
FIGURA 10 – Resultado da Consulta 10 – Pedidos realizados em março de 2026 (O AUTOR, 2026)	32
FIGURA 11 – Resultado da Consulta 11 – Total de clientes cadastrados (O AUTOR, 2026)	33
FIGURA 12 – Resultado da Consulta 12 – Total de produtos cadastrados (O AUTOR, 2026)	33
FIGURA 13 – Resultado da Consulta 13 – Produtos ordenados por preço decrescente (O AUTOR, 2026)	34
FIGURA 14 – Resultado da Consulta 14 – Quantidade total de itens em estoque (O AUTOR, 2026)	34

FIGURA 15 – Resultado da Consulta 15 – Faturamento total dos pedidos (O AUTOR, 2026)	35
FIGURA 16 – Resultado da Consulta 16 – Média de preço dos produtos (O AUTOR, 2026)	35
FIGURA 17 – Resultado da Consulta 17 – Faturamento efetivo — apenas pedidos pagos (O AUTOR, 2026)	36
FIGURA 18 – Resultado da Consulta 18 – Ticket médio dos pedidos (O AUTOR, 2026)	37
FIGURA 19 – Resultado da Consulta 19 – Pedidos agrupados por status (O AUTOR, 2026)	37
FIGURA 20 – Resultado da Consulta 20 – Total gasto por cliente (O AUTOR, 2026)	38
FIGURA 21 – Resultado da Consulta 21 – Produtos com preço acima da média (O AUTOR, 2026)	39
FIGURA 22 – Resultado da Consulta 22 – Quantidade de produtos por categoria (O AUTOR, 2026)	40
FIGURA 23 – Resultado da Consulta 23 – Ranking de clientes por valor gasto com nome (O AUTOR, 2026)	41
FIGURA 24 – Resultado da Consulta 24 – Total e receita gerada por produto (O AUTOR, 2026)	41
FIGURA 25 – Resultado da Consulta 25 – Itens de pedido com preço acima de R\$ 1.000 (O AUTOR, 2026)	42
FIGURA 26 – Resultado da Consulta 26 – Pedidos com nome do cliente (O AUTOR, 2026)	43
FIGURA 27 – Resultado da Consulta 27 – Produtos com nome da categoria (O AUTOR, 2026)	44
FIGURA 28 – Resultado da Consulta 28 – Produtos com nome do fornecedor (O AUTOR, 2026)	45
FIGURA 29 – Resultado da Consulta 29 – Clientes sem pedidos registrados (O AUTOR, 2026)	46

FIGURA 30 – Resultado da Consulta 30 – Visão consolidada dos pedidos
(O AUTOR, 2026)

47

LISTA DE ALGORITMOS

ALGORITMO 1 – Criação das Coleções (O AUTOR, 2026)	22
ALGORITMO 2 – Inserção de Clientes (O AUTOR, 2026)	22
ALGORITMO 3 – Inserção de Categorias (O AUTOR, 2026)	23
ALGORITMO 4 – Inserção de Fornecedores (O AUTOR, 2026)	23
ALGORITMO 5 – Inserção de Produtos (O AUTOR, 2026)	24
ALGORITMO 6 – Inserção de Estoque (O AUTOR, 2026)	24
ALGORITMO 7 – Inserção de Pedidos (O AUTOR, 2026)	26
ALGORITMO 8 – Consulta 01 – Clientes cadastrados (O AUTOR, 2026)	26
ALGORITMO 9 – Consulta 02 – Produtos acima de R\$ 500 (O AUTOR, 2026)	27
ALGORITMO 10 – Consulta 03 – Pedidos com status Pago (O AUTOR, 2026)	27
ALGORITMO 11 – Consulta 04 – Estoque abaixo de 20 unidades (O AUTOR, 2026)	28
ALGORITMO 12 – Consulta 05 – Clientes cadastrados em janeiro (O AUTOR, 2026)	29
ALGORITMO 13 – Consulta 06 – Pagamento aprovado via cartão de crédito (O AUTOR, 2026)	29
ALGORITMO 14 – Consulta 07 – Pedidos pendentes ou cancelados (O AUTOR, 2026)	30
ALGORITMO 15 – Consulta 08 – Produto mais caro (O AUTOR, 2026)	31
ALGORITMO 16 – Consulta 09 – Produto mais barato (O AUTOR, 2026)	31
ALGORITMO 17 – Consulta 10 – Pedidos realizados em março (O AUTOR, 2026)	32
ALGORITMO 18 – Consulta 11 – Total de clientes cadastrados (O AUTOR, 2026)	32

ALGORITMO 19 – Consulta 12 – Total de produtos cadastrados (O AUTOR, 2026)	33
ALGORITMO 20 – Consulta 13 – Produtos ordenados por preço (O AUTOR, 2026)	33
ALGORITMO 21 – Consulta 14 – Quantidade total em estoque (O AUTOR, 2026)	34
ALGORITMO 22 – Consulta 15 – Faturamento total dos pedidos (O AUTOR, 2026)	35
ALGORITMO 23 – Consulta 16 – Média de preço dos produtos (O AUTOR, 2026)	35
ALGORITMO 24 – Consulta 17 – Faturamento efetivo (pedidos pagos) (O AUTOR, 2026)	36
ALGORITMO 25 – Consulta 18 – Ticket médio dos pedidos (O AUTOR, 2026)	36
ALGORITMO 26 – Consulta 19 – Pedidos agrupados por status (O AUTOR, 2026)	37
ALGORITMO 27 – Consulta 20 – Total gasto por cliente (O AUTOR, 2026)	38
ALGORITMO 28 – Consulta 21 – Produtos acima da média de preço (O AUTOR, 2026)	38
ALGORITMO 29 – Consulta 22 – Produtos por categoria (O AUTOR, 2026)	39
ALGORITMO 30 – Consulta 23 – Ranking de clientes por valor gasto (O AUTOR, 2026)	40
ALGORITMO 31 – Consulta 24 – Total e receita por produto (O AUTOR, 2026)	41
ALGORITMO 32 – Consulta 25 – Itens com preço acima de R\$ 1.000 (O AUTOR, 2026)	42
ALGORITMO 33 – Consulta 26 – Pedidos com nome do cliente (O AUTOR, 2026)	43

ALGORITMO 34 – Consulta 27 – Produtos com nome da categoria (O AUTOR, 2026)	44
ALGORITMO 35 – Consulta 28 – Produtos com nome do fornecedor (O AUTOR, 2026)	45
ALGORITMO 36 – Consulta 29 – Clientes sem pedidos (O AUTOR, 2026)	46
ALGORITMO 37 – Consulta 30 – Visão consolidada dos pedidos (O AUTOR, 2026)	46

SUMÁRIO

1	INTRODUÇÃO	11
1.1	Objetivos	11
1.2	Justificativa	12
1.3	Aspectos Metodológicos	12
1.4	Aporte Teórico	13
2	FUNDAMENTAÇÃO TEÓRICA	14
2.1	Bancos de Dados Não Relacionais	14
2.2	MongoDB e o Modelo de Documentos	15
3	METODOLOGIA	17
3.1	Considerações Iniciais	17
3.2	Levantamento de Requisitos	17
3.3	Regras de Negócio	18
3.4	Estratégia de Modelagem	18
3.5	Ferramentas Utilizadas	19
4	RESULTADOS OBTIDOS	20
4.1	Modelagem dos Documentos	20
4.2	Criação das Coleções e Inserção de Dados	22
4.3	Consultas MQL	26
5	CONCLUSÃO	48
	REFERÊNCIAS	50

1 INTRODUÇÃO

O crescimento acelerado do comércio eletrônico nas últimas décadas produziu uma demanda crescente por sistemas capazes de lidar com grandes volumes de dados heterogêneos, alta disponibilidade e escalabilidade horizontal. Nesse contexto, os bancos de dados relacionais, embora robustos e amplamente consolidados no mercado, apresentam limitações estruturais quando o volume de dados cresce de forma não linear ou quando a estrutura dos dados se modifica com frequência ao longo do tempo. Os bancos de dados não relacionais, conhecidos como NoSQL, surgem como alternativa para cenários nos quais flexibilidade de esquema, escalabilidade e desempenho em leitura são fatores prioritários.

O presente trabalho tem como ponto de partida o projeto de banco de dados relacional desenvolvido anteriormente na disciplina de Banco de Dados 2, no qual foi implementado um sistema de e-commerce utilizando MySQL. A proposta agora é transpor esse mesmo sistema para o paradigma NoSQL, utilizando o MongoDB como sistema gerenciador de banco de dados e o modelo orientado a documentos como base para a nova modelagem.

Essa transição não consiste em uma simples conversão de tabelas para coleções. Ela exige repensar a forma como os dados se relacionam, como são consultados com maior frequência e quais informações devem ser mantidas juntas em um mesmo documento para maximizar o desempenho das operações mais comuns do sistema. As decisões de modelagem tomadas neste projeto são, portanto, justificadas com base nas regras de negócio e nos padrões de acesso aos dados.

1.1 Objetivos

O objetivo geral deste trabalho é desenvolver um banco de dados não relacional para um sistema de e-commerce utilizando MongoDB, com modelagem orientada a documentos e implementação das principais operações de consulta e manipulação de dados por meio da MongoDB Query Language.

Os objetivos específicos são:

- a) descrever os conceitos fundamentais dos bancos de dados não relacionais e seus principais modelos de dados;
- b) apresentar o MongoDB como sistema gerenciador NoSQL e caracterizar o modelo de dados orientado a documentos;
- c) definir a estratégia de modelagem dos documentos com base nas regras de negócio do sistema de e-commerce;
- d) implementar as coleções MongoDB com inserção de dados fictícios representativos;
- e) desenvolver trinta consultas MQL cobrindo diferentes níveis de complexidade, com análise interpretativa dos resultados obtidos;
- f) avaliar as diferenças entre a abordagem relacional e a abordagem de documentos para o mesmo domínio de aplicação.

1.2 Justificativa

A escolha do MongoDB como objeto de estudo justifica-se por sua ampla adoção no mercado de tecnologia, sua maturidade como plataforma NoSQL orientada a documentos e sua relevância no contexto de sistemas de e-commerce modernos. Empresas de grande porte como eBay, Adobe e Forbes utilizam MongoDB em produção para gerenciamento de catálogos de produtos, perfis de usuários e dados transacionais, o que demonstra a aplicabilidade prática do conhecimento desenvolvido neste trabalho.

Do ponto de vista acadêmico, o trabalho justifica-se pela oportunidade de comparar, sobre o mesmo domínio de aplicação, duas abordagens distintas de modelagem de dados — a relacional e a orientada a documentos — permitindo compreender com mais profundidade as vantagens, limitações e decisões de projeto de cada paradigma.

1.3 Aspectos Metodológicos

Este trabalho foi desenvolvido por meio de pesquisa bibliográfica e aplicação prática dos conceitos de modelagem NoSQL. A base teórica foi construída a partir de obras de referência sobre bancos de dados não relacionais e da documentação oficial do MongoDB.

O cenário adotado é o de um sistema de e-commerce responsável pelo gerenciamento de clientes, produtos, categorias, fornecedores, controle de estoque, pedidos com seus respectivos itens e registros de pagamento. Esse cenário foi herdado do projeto relacional anterior, o que permite uma análise comparativa direta entre os dois paradigmas.

A estratégia de modelagem adotada foi híbrida: nos casos em que dados são consultados predominantemente em conjunto — como os itens de um pedido em relação ao pedido em si — optou-se pelo embedding de subdocumentos. Nos casos em que os dados são reutilizados por múltiplos documentos e sua atualização centralizada é importante — como categorias, fornecedores e produtos referenciados em pedidos — optou-se pelo uso de referências entre coleções.

1.4 Aporte Teórico

Os bancos de dados NoSQL representam uma família de sistemas de armazenamento de dados que abandona o modelo relacional em favor de estruturas mais flexíveis. Segundo Sadalage e Fowler (2013), o termo NoSQL não implica ausência total de linguagem de consulta, mas sim a não adesão ao modelo relacional e ao SQL como mecanismo principal de interação. Esses sistemas surgiram em resposta às demandas de escalabilidade horizontal das grandes plataformas web da primeira década dos anos 2000 — como Google, Amazon e Facebook — que encontraram limitações nos bancos de dados relacionais tradicionais ao lidar com volumes de dados na ordem de petabytes.

O MongoDB, em particular, adota o modelo orientado a documentos, no qual os dados são armazenados em documentos BSON — formato binário derivado do JSON. Cada documento pode conter estruturas aninhadas e arrays, eliminando a necessidade de junções para recuperar entidades relacionadas que são frequentemente consultadas juntas. Essa característica representa uma diferença fundamental em relação ao modelo relacional, onde a normalização impõe a separação das informações em tabelas distintas para reduzir redundância.

2 FUNDAMENTAÇÃO TEÓRICA

Esta seção apresenta os conceitos teóricos fundamentais para a compreensão do projeto desenvolvido, abordando conteúdos relacionados a bancos de dados não relacionais, seus modelos de dados e ao MongoDB como sistema gerenciador escolhido.

2.1 Bancos de Dados Não Relacionais

Os bancos de dados não relacionais, também chamados de NoSQL, constituem uma categoria ampla de sistemas de gerenciamento de dados que não adotam o modelo relacional como base estrutural. Diferentemente dos bancos de dados relacionais, que organizam os dados em tabelas com esquemas rígidos e utilizam SQL como linguagem padrão de consulta, os bancos NoSQL oferecem modelos de dados alternativos que permitem maior flexibilidade estrutural, escalabilidade horizontal e desempenho otimizado para padrões específicos de acesso.

Essa categoria abrange quatro grandes modelos de dados, descritos a seguir.

O modelo de chave-valor é o mais simples entre os modelos NoSQL. Os dados são armazenados como pares de chave e valor, em que a chave é um identificador único e o valor pode ser qualquer tipo de dado — desde uma string simples até um objeto binário complexo. Esse modelo é especialmente eficiente para operações de leitura e escrita de alta velocidade, sendo amplamente utilizado em sistemas de cache e gerenciamento de sessões. Redis e Amazon DynamoDB são exemplos representativos.

O modelo de colunas largas organiza os dados em tabelas, mas de forma fundamentalmente diferente do modelo relacional: cada linha pode conter um conjunto diferente de colunas, agrupadas em famílias. Esse modelo é adequado para cargas de trabalho analíticas com grandes volumes de dados e consultas que envolvem um número limitado de colunas sobre muitas linhas. Apache Cassandra e HBase são exemplos representativos.

O modelo de documentos armazena os dados em documentos semiestruturados — geralmente no formato JSON ou em variantes binárias como BSON. Cada documento é uma unidade autônoma de dados que pode conter

campos simples, arrays e subdocumentos aninhados. A ausência de esquema fixo permite que documentos de uma mesma coleção tenham estruturas distintas, conferindo grande flexibilidade para evolução do modelo de dados ao longo do tempo. MongoDB e CouchDB são os representantes mais conhecidos.

O modelo de grafos representa os dados como nós e arestas de um grafo, sendo especialmente adequado para domínios em que os relacionamentos entre entidades são tão importantes quanto os próprios dados. Redes sociais, sistemas de recomendação e análise de fraudes são casos de uso típicos. Neo4j e Amazon Neptune são exemplos de SGBDs baseados em grafos.

2.2 MongoDB e o Modelo de Documentos

O MongoDB é um sistema gerenciador de banco de dados não relacional orientado a documentos, desenvolvido pela MongoDB Inc. e disponibilizado inicialmente em 2009. É escrito em C++ e distribuído sob licença de código aberto, com versões Community e Enterprise. Atualmente, é o banco de dados NoSQL mais amplamente adotado no mercado, ocupando consistentemente as primeiras posições nos rankings de popularidade de bancos de dados.

No MongoDB, os dados são organizados em coleções — equivalentes conceituais às tabelas do modelo relacional, mas sem imposição de esquema fixo. Cada coleção armazena documentos no formato BSON (Binary JSON), que estende o JSON com tipos de dados adicionais como datas, inteiros de 64 bits e identificadores únicos do tipo ObjectId. Cada documento possui automaticamente um campo `_id` que funciona como chave primária, podendo ser gerado automaticamente pelo sistema ou fornecido pela aplicação.

A principal diferença estrutural em relação ao modelo relacional está na capacidade de embedding: um documento pode conter outros documentos aninhados e arrays de documentos, permitindo representar relacionamentos sem recorrer a junções. Isso torna as operações de leitura muito eficientes para os padrões de acesso mais comuns, pois todas as informações necessárias para uma operação típica podem ser recuperadas com uma única consulta a um único documento.

A MongoDB Query Language (MQL) oferece um conjunto rico de operadores para filtragem, projeção, ordenação, agrupamento e transformação de dados. O

framework de agregação, baseado no conceito de pipeline, permite encadear múltiplos estágios de processamento — como `$match` para filtragem, `$group` para agrupamento, `$lookup` para junção entre coleções, `$unwind` para expansão de arrays e `$project` para projeção e transformação de campos — construindo assim consultas analíticas sofisticadas de forma expressiva e legível.

3 METODOLOGIA

Nesta seção são descritos os procedimentos adotados para análise, modelagem e implementação do banco de dados não relacional do sistema de e-commerce com MongoDB.

3.1 Considerações Iniciais

O projeto foi desenvolvido com o objetivo de modelar e implementar um sistema de banco de dados não relacional para um e-commerce utilizando MongoDB. O ponto de partida foi o banco de dados relacional desenvolvido anteriormente com MySQL, que serviu como referência para o levantamento de requisitos e regras de negócio. A transposição para o modelo de documentos exigiu revisão das decisões de modelagem, uma vez que as estratégias de normalização aplicadas no modelo relacional não se aplicam diretamente ao paradigma NoSQL.

A escolha do MongoDB como SGBD justifica-se por três razões principais. Primeiro, sua adoção é massiva no mercado de e-commerce, com casos de uso documentados em empresas como eBay, que utiliza MongoDB para gerenciar catálogos de produtos com estruturas variáveis. Segundo, o modelo de documentos é particularmente adequado para domínios como e-commerce, onde entidades como pedidos e produtos possuem atributos variáveis e são frequentemente consultadas com seus dados relacionados em uma única operação. Terceiro, o MongoDB oferece um ecossistema maduro, com ferramentas como MongoDB Atlas, MongoDB Compass e drivers para todas as principais linguagens de programação, o que facilita sua adoção em projetos reais.

3.2 Levantamento de Requisitos

As necessidades funcionais identificadas para o sistema de e-commerce são as mesmas do projeto relacional anterior, adaptadas para o paradigma de documentos:

- a) cadastro e consulta de clientes com seus dados pessoais e de contato;

- b) cadastro de produtos com descrição, preço, categoria e fornecedor associado;
- c) controle de estoque por produto;
- d) registro de pedidos associados a clientes, contendo os produtos comprados, quantidades e subtotais;
- e) registro de pagamentos associados aos pedidos;
- f) associação de produtos a categorias e fornecedores.

3.3 Regras de Negócio

As regras de negócio do sistema, originalmente mapeadas para o modelo relacional, foram revisadas e adaptadas para orientar as decisões de modelagem no MongoDB:

- a) um cliente pode realizar zero ou múltiplos pedidos. Seus dados de contato são únicos por CPF e e-mail;
- b) cada pedido pertence a exatamente um cliente e deve conter ao menos um item de produto;
- c) os itens de um pedido registram o identificador do produto, a quantidade e o subtotal calculado no momento da compra. O preço do produto pode variar ao longo do tempo, por isso o valor do item é registrado de forma imutável dentro do pedido;
- d) cada produto pertence a exatamente uma categoria e possui exatamente um fornecedor principal;
- e) o estoque é controlado por produto, com registro da quantidade disponível;
- f) um pedido pode ter zero ou múltiplos registros de pagamento, permitindo o rastreamento de tentativas pendentes, recusadas e aprovadas;
- g) categorias e fornecedores podem existir sem produtos vinculados.

3.4 Estratégia de Modelagem

A estratégia de modelagem adotada foi híbrida, combinando embedding e referências conforme a natureza de cada relacionamento.

O embedding foi adotado para os itens de pedido, que foram embutidos diretamente nos documentos de pedido. Essa decisão fundamenta-se no fato de que itens de pedido são sempre consultados em conjunto com o pedido ao qual pertencem, nunca de forma isolada. Além disso, os valores de subtotal e quantidade são registros históricos imutáveis do momento da compra, o que elimina a necessidade de atualização centralizada. Também foram embutidos no documento de pedido os campos de pagamento, uma vez que a operação de consulta de um pedido normalmente exige conhecer seu status de pagamento.

As referências foram adotadas para os relacionamentos entre produtos e suas categorias e fornecedores. Categorias e fornecedores são entidades reutilizadas por múltiplos produtos, e sua atualização deve ser refletida em todos os produtos associados sem necessidade de varredura de toda a coleção. Da mesma forma, os pedidos referenciam clientes por identificador, pois o perfil completo do cliente é uma entidade independente com ciclo de vida próprio.

O campo de preço é armazenado diretamente no item do pedido no momento da compra, garantindo rastreabilidade histórica mesmo que o preço do produto seja alterado posteriormente. Trata-se de uma desnormalização intencional e justificada pelo requisito de integridade histórica de dados transacionais.

3.5 Ferramentas Utilizadas

As seguintes ferramentas foram utilizadas no desenvolvimento do projeto:

- a) MongoDB Community Server, versão 7.0;
- b) MongoDB Shell (mongosh) para execução de comandos e consultas;
- c) MongoDB Compass para visualização e exploração dos documentos;
- d) JavaScript como linguagem de scripting para inserção de dados e execução de pipelines de agregação.

4 RESULTADOS OBTIDOS

Os resultados apresentados nesta seção correspondem às etapas de modelagem e implementação do banco de dados não relacional, incluindo a estrutura dos documentos de cada coleção, os comandos de criação e inserção de dados e as trinta consultas desenvolvidas com a MongoDB Query Language.

4.1 Modelagem dos Documentos

A seguir é apresentada a estrutura dos documentos de cada coleção do sistema, equivalente ao modelo lógico do projeto relacional. A modelagem reflete a estratégia híbrida descrita na seção de metodologia.

Coleção: clientes

```
{
  "_id": ObjectId(),
  "nome": "string",
  "email": "string",
  "telefone": "string",
  "cpf": "string",
  "data_cadastro": ISODate()
}
```

Coleção: categorias

```
{
  "_id": ObjectId(),
  "nome_categoria": "string",
  "descricao_categoria": "string"
}
```

Coleção: fornecedores

```
{
  "_id": ObjectId(),
  "nome": "string",
  "razao_social": "string",
  "cnpj": "string",
  "telefone": "string"
}
```

Coleção: produtos

```
{
  "_id": ObjectId(),
  "nome": "string",
  "descricao": "string",
  "preco": NumberDecimal(),
  "id_categoria": ObjectId(),    // referência à coleção categorias
  "id_fornecedor": ObjectId()   // referência à coleção fornecedores
}
```

Coleção: estoque

```
{
  "_id": ObjectId(),
  "id_produto": ObjectId(),      // referência à coleção produtos
  "quantidade": NumberInt(),
  "localizacao": "string"
}
```

Coleção: pedidos (com itens e pagamentos embutidos)

```
{
  "_id": ObjectId(),
  "id_cliente": ObjectId(),      // referência à coleção clientes
  "data_pedido": ISODate(),
  "valor_total": NumberDecimal(),
  "status": "string",
  "itens": [
    {
      "id_produto": ObjectId(),
      "nome_produto": "string",
      "quantidade": NumberInt(),
      "preco_unitario": NumberDecimal(),
      "subtotal": NumberDecimal()
    }
  ],
  "pagamentos": [
    {
      "tipo_pagamento": "string",
      "status_pagamento": "string",
      "data_pagamento": ISODate(),
      "valor_pago": NumberDecimal()
    }
  ]
}
```

A decisão de embutir os itens e os pagamentos dentro do documento de pedido fundamenta-se no padrão de acesso predominante: toda consulta a um pedido requer simultaneamente seus itens e seu status de pagamento. No modelo relacional, essa operação demandaria junções entre quatro tabelas distintas. No modelo de documentos, essas informações são recuperadas em uma única

operação de leitura. O campo `nome_produto` foi incluído no subdocumento de item por razão de rastreabilidade histórica: caso o nome do produto seja alterado posteriormente, o pedido original preserva o nome vigente no momento da compra.

4.2 Criação das Coleções e Inserção de Dados

A seguir são apresentados os comandos JavaScript utilizados para criar o banco de dados e as coleções, bem como para inserir os dados fictícios que serviram de base para as consultas desenvolvidas.

Criação das Coleções

```
use ecommerce_nosql

db.createCollection("clientes")
db.createCollection("categorias")
db.createCollection("fornecedores")
db.createCollection("produtos")
db.createCollection("estoque")
db.createCollection("pedidos")
```

Algoritmo 1 – Criação das Coleções (O AUTOR, 2026)

Inserção de Clientes

```
db.clientes.insertMany([
  {
    nome: "Carlos Silva",
    email: "carlos@gmail.com",
    telefone: "11999999999",
    cpf: "111.111.111-11",
    data_cadastro: ISODate("2026-01-10")
  },
  {
    nome: "Mariana Souza",
    email: "mariana@gmail.com",
    telefone: "11988888888",
    cpf: "222.222.222-22",
    data_cadastro: ISODate("2026-01-15")
  },
  {
    nome: "Lucas Ferreira",
    email: "lucas@gmail.com",
    telefone: "11977777777",
    cpf: "333.333.333-33",
    data_cadastro: ISODate("2026-02-01")
  }
])
```

```
])
```

Algoritmo 2 – Inserção de Clientes (O AUTOR, 2026)

Inserção de Categorias

```
db.categorias.insertMany([
  { nome_categoria: "Eletrônicos", descricao_categoria: "Dispositivos
eletrônicos em geral" },
  { nome_categoria: "Informática", descricao_categoria: "Equipamentos
e periféricos de informática" },
  { nome_categoria: "Acessórios", descricao_categoria: "Acessórios
para dispositivos eletrônicos" }
])
```

Algoritmo 3 – Inserção de Categorias (O AUTOR, 2026)

Inserção de Fornecedores

```
db.fornecedores.insertMany([
  {
    nome: "Tech Distribuidora",
    razao_social: "Tech Distribuidora Ltda",
    cnpj: "11.111.111/0001-11",
    telefone: "1133333333"
  },
  {
    nome: "Global Imports",
    razao_social: "Global Imports Comércio Exterior SA",
    cnpj: "22.222.222/0001-22",
    telefone: "1144444444"
  }
])
```

Algoritmo 4 – Inserção de Fornecedores (O AUTOR, 2026)

Inserção de Produtos

```
// Recuperar IDs necessários
const catInformatica = db.categorias.findOne({ nome_categoria:
"Informática" })._id
const catEletronicos = db.categorias.findOne({ nome_categoria:
"Eletrônicos" })._id
const catAcessorios = db.categorias.findOne({ nome_categoria:
"Acessórios" })._id
const fornTech = db.fornecedores.findOne({ nome: "Tech
Distribuidora" })._id
const fornGlobal = db.fornecedores.findOne({ nome: "Global Imports"
})._id
```

```

db.produtos.insertMany([
  {
    nome: "Notebook Gamer",
    descricao: "Notebook para jogos com placa de vídeo dedicada",
    preco: NumberDecimal("4500.00"),
    id_categoria: catInformatica,
    id_fornecedor: fornTech
  },
  {
    nome: "Mouse Gamer",
    descricao: "Mouse RGB com 7 botões programáveis",
    preco: NumberDecimal("150.00"),
    id_categoria: catAcessorios,
    id_fornecedor: fornGlobal
  },
  {
    nome: "Smartphone",
    descricao: "Celular Android 256GB",
    preco: NumberDecimal("2500.00"),
    id_categoria: catEletronicos,
    id_fornecedor: fornTech
  }
])

```

Algoritmo 5 – Inserção de Produtos (O AUTOR, 2026)

Inserção de Estoque

```

const prodNotebook = db.produtos.findOne({ nome: "Notebook Gamer"
})._id
const prodMouse = db.produtos.findOne({ nome: "Mouse Gamer" })._id
const prodSmartphone = db.produtos.findOne({ nome: "Smartphone" })._id

db.estoque.insertMany([
  { id_produto: prodNotebook, quantidade: 20, localizacao:
"Prateleira A1" },
  { id_produto: prodMouse, quantidade: 50, localizacao:
"Prateleira B3" },
  { id_produto: prodSmartphone, quantidade: 15, localizacao:
"Prateleira C2" }
])

```

Algoritmo 6 – Inserção de Estoque (O AUTOR, 2026)

Inserção de Pedidos (com itens e pagamentos embutidos)

```

const clienteCarlos = db.clientes.findOne({ nome: "Carlos Silva"
})._id
const clienteMariana = db.clientes.findOne({ nome: "Mariana Souza"
})._id
const clienteLucas = db.clientes.findOne({ nome: "Lucas Ferreira"
})._id

```



```

db.pedidos.insertMany([
  {
    id_cliente: clienteCarlos,
    data_pedido: ISODate("2026-03-10"),
    valor_total: NumberDecimal("4650.00"),
    status: "Pago",
    itens: [
      {
        id_produto: prodNotebook,
        nome_produto: "Notebook Gamer",
        quantidade: 1,
        preco_unitario: NumberDecimal("4500.00"),
        subtotal: NumberDecimal("4500.00")
      },
      {
        id_produto: prodMouse,
        nome_produto: "Mouse Gamer",
        quantidade: 1,
        preco_unitario: NumberDecimal("150.00"),
        subtotal: NumberDecimal("150.00")
      }
    ],
    pagamentos: [
      {
        tipo_pagamento: "Cartão de Crédito",
        status_pagamento: "Aprovado",
        data_pagamento: ISODate("2026-03-10"),
        valor_pago: NumberDecimal("4650.00")
      }
    ]
  },
  {
    id_cliente: clienteMariana,
    data_pedido: ISODate("2026-03-15"),
    valor_total: NumberDecimal("2500.00"),
    status: "Pendente",
    itens: [
      {
        id_produto: prodSmartphone,
        nome_produto: "Smartphone",
        quantidade: 1,
        preco_unitario: NumberDecimal("2500.00"),
        subtotal: NumberDecimal("2500.00")
      }
    ],
    pagamentos: [
      {
        tipo_pagamento: "PIX",
        status_pagamento: "Pendente",
        data_pagamento: ISODate("2026-03-15"),
        valor_pago: NumberDecimal("2500.00")
      }
    ]
  },
  {
    id_cliente: clienteLucas,
    data_pedido: ISODate("2026-04-02"),
    valor_total: NumberDecimal("300.00"),
    status: "Cancelado",
    itens: [

```

```

        {
            id_produto: prodMouse,
            nome_produto: "Mouse Gamer",
            quantidade: 2,
            preco_unitario: NumberDecimal("150.00"),
            subtotal: NumberDecimal("300.00")
        }
    ],
    pagamentos: [
        {
            tipo_pagamento: "Boleto",
            status_pagamento: "Cancelado",
            data_pagamento: ISODate("2026-04-02"),
            valor_pago: NumberDecimal("0.00")
        }
    ]
}
])

```

Algoritmo 7 – Inserção de Pedidos (O AUTOR, 2026)

4.3 Consultas MQL

As trinta consultas desenvolvidas estão organizadas em quatro categorias: consultas simples de filtragem e projeção (Consultas 1 a 10), consultas com ordenação, contagem e agregação básica (Consultas 11 a 18), consultas com pipeline de agregação e agrupamento (Consultas 19 a 25) e consultas com junções entre coleções utilizando \$lookup (Consultas 26 a 30).

Categoria 1 — Consultas Simples de Filtragem e Projeção

Consulta 1 – Clientes cadastrados

Descrição: Retorna o nome e o e-mail de todos os clientes cadastrados no sistema.

```

db.clientes.find(
  {},
  { nome: 1, email: 1, _id: 0 }
)

```

Algoritmo 8 – Consulta 01 – Clientes cadastrados (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.clientes.find({}, { nome: 1, email: 1, _id: 0 })
[
  { nome: 'Carlos Silva', email: 'carlos@gmail.com' },
  { nome: 'Mariana Souza', email: 'mariana@gmail.com' },
  { nome: 'Lucas Ferreira', email: 'lucas@gmail.com' }
]
```

Figura 1 – Resultado da Consulta 01 – Clientes cadastrados (O AUTOR, 2026)

A consulta retorna os três clientes cadastrados com seus respectivos e-mails. O operador de projeção { _id: 0 } suprime o identificador interno, tornando a saída mais limpa. Do ponto de vista de negócio, essa listagem é útil para operações de CRM e envio de comunicações em massa.

Consulta 2 – Produtos acima de R\$ 500

Descrição: Retorna os produtos cujo preço é superior a R\$ 500,00.

```
db.produtos.find(
  { preco: { $gt: NumberDecimal("500.00") } },
  { nome: 1, preco: 1, _id: 0 }
)
```

Algoritmo 9 – Consulta 02 – Produtos acima de R\$ 500 (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.produtos.find({ preco: { $gt: NumberDecimal("500.00") } }, { nome: 1, preco: 1, _id: 0 })
[
  { nome: 'Notebook Gamer', preco: Decimal128('4500.00') },
  { nome: 'Smartphone', preco: Decimal128('2500.00') }
]
```

Figura 2 – Resultado da Consulta 02 – Produtos acima de R\$ 500 (O AUTOR, 2026)

São retornados o Notebook Gamer (R\$ 4.500,00) e o Smartphone (R\$ 2.500,00). O Mouse Gamer, com preço de R\$ 150,00, é corretamente excluído. Essa consulta é relevante para identificação de produtos de alto valor para campanhas de seguro ou financiamento parcelado.

Consulta 3 – Pedidos com status Pago

Descrição: Retorna todos os pedidos com status igual a Pago.

```
db.pedidos.find(
  { status: "Pago" },
  { id_cliente: 1, data_pedido: 1, valor_total: 1, status: 1, _id: 0 }
)
```

Algoritmo 10 – Consulta 03 – Pedidos com status Pago (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.pedidos.find({ status: "Pago" }, { id_cliente: 1, data_pedido: 1, valor_total: 1, status: 1, _id: 0 })
[
  {
    id_cliente: ObjectId('6a36ba6937377719f3abc114'),
    data_pedido: ISODate('2026-03-10T00:00:00.000Z'),
    valor_total: Decimal128('4650.00'),
    status: 'Pago'
  }
]
```

Figura 3 – Resultado da Consulta 03 – Pedidos com status Pago (O AUTOR, 2026)

Apenas o pedido de Carlos Silva, no valor de R\$ 4.650,00, consta com status Pago. Para a gestão financeira, essa consulta permite calcular o faturamento efetivamente recebido em um determinado período, excluindo pedidos pendentes ou cancelados.

Consulta 4 – Estoque abaixo de 20 unidades

Descrição: Retorna os produtos com quantidade em estoque inferior a 20 unidades.

```
db.estoque.find(
  { quantidade: { $lt: 20 } },
  { id_produto: 1, quantidade: 1, localizacao: 1, _id: 0 }
)
```

Algoritmo 11 – Consulta 04 – Estoque abaixo de 20 unidades (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.estoque.find({ quantidade: { $lt: 20 } }, { id_produto: 1, quantidade: 1, localizacao: 1, _id: 0 })
[
  {
    id_produto: ObjectId('6a36baba37377719f3abc11e'),
    quantidade: 15,
    localizacao: 'Prateleira C2'
  }
]
```

Figura 4 – Resultado da Consulta 04 – Estoque abaixo de 20 unidades (O AUTOR, 2026)

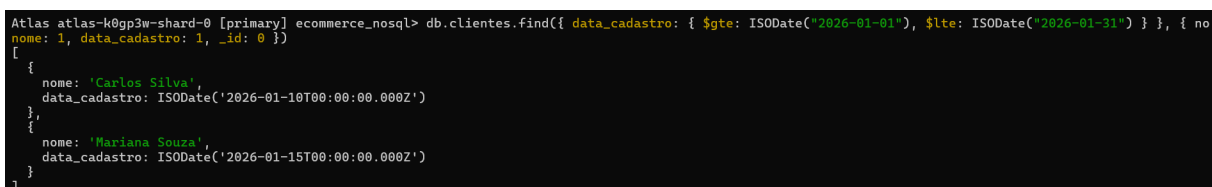
O registro do Smartphone aparece com 15 unidades disponíveis, abaixo do limiar configurado. Esse tipo de consulta é fundamental para alertas automáticos de reposição de estoque, evitando ruptura de disponibilidade para o cliente final.

Consulta 5 – Clientes cadastrados em janeiro de 2026

Descrição: Retorna os clientes cujo cadastro foi realizado no mês de janeiro de 2026.

```
db.clientes.find(
  {
    data_cadastro: {
      $gte: ISODate("2026-01-01"),
      $lte: ISODate("2026-01-31")
    }
  },
  { nome: 1, data_cadastro: 1, _id: 0 }
)
```

Algoritmo 12 – Consulta 05 – Clientes cadastrados em janeiro de 2026 (O AUTOR, 2026)



```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.clientes.find({ data_cadastro: { $gte: ISODate("2026-01-01"), $lte: ISODate("2026-01-31") } }, { no
nome: 1, data_cadastro: 1, _id: 0 })
[
  {
    nome: 'Carlos Silva',
    data_cadastro: ISODate('2026-01-10T00:00:00.000Z')
  },
  {
    nome: 'Mariana Souza',
    data_cadastro: ISODate('2026-01-15T00:00:00.000Z')
  }
]
```

Figura 5 – Resultado da Consulta 05 – Clientes cadastrados em janeiro de 2026 (O AUTOR, 2026)

Carlos Silva (10/01/2026) e Mariana Souza (15/01/2026) são retornados. Lucas Ferreira, cadastrado em fevereiro, é corretamente excluído. Essa consulta serve para análise de aquisição de novos clientes por período, métrica essencial em dashboards de marketing.

Consulta 6 – Pagamento aprovado via Cartão de Crédito

Descrição: Retorna pedidos que possuem ao menos um pagamento aprovado realizado com cartão de crédito.

```
db.pedidos.find(
  {
    pagamentos: {
      $elemMatch: {
        tipo_pagamento: "Cartão de Crédito",
        status_pagamento: "Aprovado"
      }
    }
  },
  { id_cliente: 1, valor_total: 1, status: 1, _id: 0 }
)
```

Algoritmo 13 – Consulta 06 – Pagamento aprovado via Cartão de Crédito (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.pedidos.find({ pagamentos: { $elemMatch: { tipo_pagamento: "Cartão de Crédito", status_pagamento: "Aprovado" } } }, { id_cliente: 1, valor_total: 1, status: 1, _id: 0 })
[
  {
    id_cliente: ObjectId('6a36ba6937377719f3abc114'),
    valor_total: Decimal128('4650.00'),
    status: 'Pago'
  }
]
```

Figura 6 – Resultado da Consulta 06 – Pagamento aprovado via Cartão de Crédito (O AUTOR, 2026)

O pedido de Carlos Silva é retornado. O operador \$elemMatch garante que ambas as condições — tipo e status do pagamento — sejam satisfeitas pelo mesmo elemento do array, evitando falsos positivos. Essa consulta é útil para conciliação com operadoras de cartão.

Consulta 7 – Pedidos pendentes ou cancelados

Descrição: Retorna pedidos cujo status seja Pendente ou Cancelado.

```
db.pedidos.find(
  { status: { $in: ["Pendente", "Cancelado"] } },
  { id_cliente: 1, data_pedido: 1, valor_total: 1, status: 1, _id: 0 }
)
```

Algoritmo 14 – Consulta 07 – Pedidos pendentes ou cancelados (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.pedidos.find({ status: { $in: ["Pendente", "Cancelado"] } }, { id_cliente: 1, data_pedido: 1, valor_total: 1, status: 1, _id: 0 })
[
  {
    id_cliente: ObjectId('6a36ba6937377719f3abc115'),
    data_pedido: ISODate('2026-03-15T00:00:00.000Z'),
    valor_total: Decimal128('2500.00'),
    status: 'Pendente'
  },
  {
    id_cliente: ObjectId('6a36ba6937377719f3abc116'),
    data_pedido: ISODate('2026-04-02T00:00:00.000Z'),
    valor_total: Decimal128('300.00'),
    status: 'Cancelado'
  }
]
```

Figura 7 – Resultado da Consulta 07 – Pedidos pendentes ou cancelados (O AUTOR, 2026)

São retornados o pedido de Mariana Souza (Pendente) e o pedido de Lucas Ferreira (Cancelado). Operacionalmente, esses pedidos demandam ação: o pendente requer acompanhamento de pagamento e o cancelado pode indicar abandono de carrinho ou falha na transação.

Consulta 8 – Produto mais caro

Descrição: Retorna o produto com o maior preço cadastrado.

```
db.produtos.find(
  {},
  { nome: 1, preco: 1, _id: 0 }
).sort({ preco: -1 }).limit(1)
```

Algoritmo 15 – Consulta 08 – Produto mais caro (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.produtos.find({}, { nome: 1, preco: 1, _id: 0 }).sort({ preco: -1 }).limit(1)
[ { nome: 'Notebook Gamer', preco: Decimal128('4588.88') } ]
```

Figura 8 – Resultado da Consulta 08 – Produto mais caro (O AUTOR, 2026)

O Notebook Gamer, com R\$ 4.500,00, é retornado como o produto de maior valor. Para o e-commerce, essa consulta informa o posicionamento da loja no segmento premium e subsidia decisões sobre destaque na vitrine principal.

Consulta 9 – Produto mais barato

Descrição: Retorna o produto com o menor preço cadastrado.

```
db.produtos.find(
  {},
  { nome: 1, preco: 1, _id: 0 }
).sort({ preco: 1 }).limit(1)
```

Algoritmo 16 – Consulta 09 – Produto mais barato (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.produtos.find({}, { nome: 1, preco: 1, _id: 0 }).sort({ preco: 1 }).limit(1)
[ { nome: 'Mouse Gamer', preco: Decimal128('158.88') } ]
```

Figura 9 – Resultado da Consulta 09 – Produto mais barato (O AUTOR, 2026)

O Mouse Gamer, com R\$ 150,00, é retornado. Produtos de menor valor são frequentemente utilizados em estratégias de entrada — como ofertas relâmpago e produtos de baixo custo que incentivam o primeiro acesso ao e-commerce.

Consulta 10 – Pedidos realizados em março de 2026

Descrição: Retorna todos os pedidos realizados no mês de março de 2026.

```
db.pedidos.find(
  {
    data_pedido: {
      $gte: ISODate("2026-03-01"),
      $lte: ISODate("2026-03-31")
    }
  },
  { id_cliente: 1, data_pedido: 1, valor_total: 1, status: 1, _id: 0 }
)
```

Algoritmo 17 – Consulta 10 – Pedidos realizados em março de 2026 (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.pedidos.find({ data_pedido: { $gte: ISODate("2026-03-01"), $lte: ISODate("2026-03-31") } }, { id_cliente: 1, data_pedido: 1, valor_total: 1, status: 1, _id: 0 })
[
  {
    id_cliente: ObjectId('6a36ba6937377719f3abc114'),
    data_pedido: ISODate('2026-03-10T00:00:00.000Z'),
    valor_total: Decimal128('4650.00'),
    status: 'Pago'
  },
  {
    id_cliente: ObjectId('6a36ba6937377719f3abc115'),
    data_pedido: ISODate('2026-03-15T00:00:00.000Z'),
    valor_total: Decimal128('2500.00'),
    status: 'Pendente'
  }
]
```

Figura 10 – Resultado da Consulta 10 – Pedidos realizados em março de 2026 (O AUTOR, 2026)

Os dois primeiros pedidos — de Carlos Silva e Mariana Souza — são retornados, ambos com datas em março. O pedido de Lucas Ferreira, de abril, é excluído corretamente. Essa consulta representa o tipo de corte temporal usado em relatórios mensais de vendas.

Categoria 2 — Consultas com Ordenação, Contagem e Agregação Básica

Consulta 11 – Total de clientes cadastrados

Descrição: Conta o número total de clientes cadastrados no sistema.

```
db.clientes.countDocuments({})
```

Algoritmo 18 – Consulta 11 – Total de clientes cadastrados (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.clientes.countDocuments({})
3
```


Figura 11 – Resultado da Consulta 11 – Total de clientes cadastrados (O AUTOR, 2026)

O sistema retorna o valor 3, correspondente aos três clientes inseridos. Essa métrica é um KPI fundamental para acompanhar o crescimento da base de usuários ao longo do tempo.

Consulta 12 – Total de produtos cadastrados

Descrição: Conta o número total de produtos cadastrados.

```
db.produtos.countDocuments({})
```

Algoritmo 19 – Consulta 12 – Total de produtos cadastrados (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.produtos.countDocuments({})
3
```

Figura 12 – Resultado da Consulta 12 – Total de produtos cadastrados (O AUTOR, 2026)

São retornados 3 produtos. Em um sistema com catálogo extenso, essa consulta é complementada por filtros de categoria ou fornecedor para produzir relatórios segmentados.

Consulta 13 – Produtos ordenados por preço decrescente

Descrição: Lista todos os produtos em ordem decrescente de preço.

```
db.produtos.find(
  {},
  { nome: 1, preco: 1, _id: 0 }
).sort({ preco: -1 })
```

Algoritmo 20 – Consulta 13 – Produtos ordenados por preço decrescente (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.produtos.find({}, { nome: 1, preco: 1, _id: 0 }).sort({ preco: -1 })
[
  { nome: 'Notebook Gamer', preco: Decimal128('4500.00') },
  { nome: 'Smartphone', preco: Decimal128('2500.00') },
  { nome: 'Mouse Gamer', preco: Decimal128('150.00') }
]
```

Figura 13 – Resultado da Consulta 13 – Produtos ordenados por preço decrescente
(O AUTOR, 2026)

O resultado exibe Notebook Gamer (R\$ 4.500,00), Smartphone (R\$ 2.500,00) e Mouse Gamer (R\$ 150,00) nessa ordem. A ordenação decrescente é o padrão mais comum em vitrines quando o cliente filtra por maior preço.

Consulta 14 – Quantidade total de itens em estoque

Descrição: Soma as quantidades de todos os produtos registrados no estoque.

```
db.estoque.aggregate([
  {
    $group: {
      _id: null,
      total_estoque: { $sum: "$quantidade" }
    }
  }
])
```

Algoritmo 21 – Consulta 14 – Quantidade total de itens em estoque (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.estoque.aggregate([{$group: { _id: null, total_estoque: { $sum: "$quantidade" } } }])
[ { _id: null, total_estoque: 85 } ]
```

Figura 14 – Resultado da Consulta 14 – Quantidade total de itens em estoque (O AUTOR, 2026)

O total retornado é 85 unidades (20 + 50 + 15). Essa métrica global de estoque é utilizada em relatórios logísticos para dimensionar espaço de armazenagem e planejamento de reposição.

Consulta 15 – Faturamento total dos pedidos

Descrição: Soma o valor total de todos os pedidos registrados, independentemente do status.

```
db.pedidos.aggregate([
  {
```

```

    $group: {
      _id: null,
      faturamento_total: { $sum: "$valor_total" }
    }
  }
])

```

Algoritmo 22 – Consulta 15 – Faturamento total dos pedidos (O AUTOR, 2026)

```

Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.pedidos.aggregate([{$group: { _id: null, faturamento_total: { $sum: "$valor_total" } } }])
[ { _id: null, faturamento_total: Decimal128('7450.00') } ]

```

Figura 15 – Resultado da Consulta 15 – Faturamento total dos pedidos (O AUTOR, 2026)

O faturamento bruto total é R\$ 7.450,00. Note que esse valor inclui pedidos em todos os status. Para uma visão de faturamento efetivo, é necessário filtrar apenas pedidos com status Pago, o que é demonstrado na Consulta 17.

Consulta 16 – Média de preço dos produtos

Descrição: Calcula a média de preço dos produtos cadastrados.

```

db.produtos.aggregate([
  {
    $group: {
      _id: null,
      media_preco: { $avg: "$preco" }
    }
  }
])

```

Algoritmo 23 – Consulta 16 – Média de preço dos produtos (O AUTOR, 2026)

```

Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.produtos.aggregate([{$group: { _id: null, media_preco: { $avg: "$preco" } } }])
[ { _id: null, media_preco: Decimal128('2383.3333333333333333333333333333') } ]

```

Figura 16 – Resultado da Consulta 16 – Média de preço dos produtos (O AUTOR, 2026)

A média retornada é aproximadamente R\$ 2.383,33. Esse valor pode servir como referência para análise do posicionamento de preço do catálogo e identificação de produtos muito abaixo ou acima da média.

Figura 18 – Resultado da Consulta 18 – Ticket médio dos pedidos (O AUTOR, 2026)

A média retornada é aproximadamente R\$ 2.483,33. Essa métrica, conhecida no varejo como ticket médio, é um dos principais indicadores de saúde de um e-commerce — seu acompanhamento ao longo do tempo indica se as estratégias de upsell e cross-sell estão sendo efetivas.

Categoria 3 — Consultas com Pipeline de Agregação e Agrupamento

Consulta 19 – Pedidos agrupados por status

Descrição: Conta quantos pedidos existem para cada status possível.

```
db.pedidos.aggregate([
  {
    $group: {
      _id: "$status",
      quantidade: { $sum: 1 }
    }
  },
  { $sort: { quantidade: -1 } }
])
```

Algoritmo 26 – Consulta 19 – Pedidos agrupados por status (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.pedidos.aggregate([{$group: { _id: "$status", quantidade: { $sum: 1 } } }, { $sort: { quantidade: -1 } }])
[
  { _id: 'Pago', quantidade: 1 },
  { _id: 'Pendente', quantidade: 1 },
  { _id: 'Cancelado', quantidade: 1 }
]
```

Figura 19 – Resultado da Consulta 19 – Pedidos agrupados por status (O AUTOR, 2026)

O resultado apresenta um pedido por status — Pago, Pendente e Cancelado. Em um sistema com volume real de pedidos, essa consulta alimenta painéis operacionais que monitoram a saúde do funil de pagamento e a taxa de conversão.

Consulta 20 – Total gasto por cliente

Descrição: Agrupa os pedidos por cliente e soma o valor total gasto por cada um.

```
db.pedidos.aggregate([
  {
    $group: {
      _id: "$id_cliente",
      total_gasto: { $sum: "$valor_total" },
      quantidade_pedidos: { $sum: 1 }
    },
    { $sort: { total_gasto: -1 } }
  ])

```

Algoritmo 27 – Consulta 20 – Total gasto por cliente (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.pedidos.aggregate([{$group: { _id: "$id_cliente", total_gasto: { $sum: "$valor_total" }, quantidade_pedidos: { $sum: 1 } }}, {$sort: { total_gasto: -1 } }])
[
  {
    _id: ObjectId('6a36ba6937377719f3abc114'),
    total_gasto: Decimal128('4650.00'),
    quantidade_pedidos: 1
  },
  {
    _id: ObjectId('6a36ba6937377719f3abc115'),
    total_gasto: Decimal128('2500.00'),
    quantidade_pedidos: 1
  },
  {
    _id: ObjectId('6a36ba6937377719f3abc116'),
    total_gasto: Decimal128('300.00'),
    quantidade_pedidos: 1
  }
]

```

Figura 20 – Resultado da Consulta 20 – Total gasto por cliente (O AUTOR, 2026)

Carlos Silva aparece com o maior volume — R\$ 4.650,00 em um único pedido — seguido por Mariana Souza (R\$ 2.500,00) e Lucas Ferreira (R\$ 300,00). Essa consulta é a base para programas de fidelidade e segmentação de clientes por valor de vida (LTV).

Consulta 21 – Produtos com preço acima da média

Descrição: Utiliza um pipeline de dois estágios para calcular a média e filtrar os produtos que a superam.

```
db.produtos.aggregate([
  { $group: { _id: null, media: { $avg: "$preco" } } },
  {
    $lookup: {
      from: "produtos",
      let: { media: "$media" },
      pipeline: [
        { $match: { $expr: { $gt: ["$preco", "$$media"] } } },
        { $project: { nome: 1, preco: 1, _id: 0 } }
      ],
      as: "produtos_acima_da_media"
    }
  },
  { $project: { _id: 0, media: 1, produtos_acima_da_media: 1 } }
])

```



```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.produtos.aggregate([{$group: { _id: "$id_categoria", quantidade_produtos: { $sum: 1 } } }, {$lookup: { from: "categorias", localField: "_id", foreignField: "_id", as: "categoria" } }, {$unwind: "$categoria" }, {$project: { _id: 0, nome_categoria: "$categoria.nome_categoria", quantidade_produtos: 1 } }, {$sort: { quantidade_produtos: -1 } }])
[
  { quantidade_produtos: 1, nome_categoria: 'Informática' },
  { quantidade_produtos: 1, nome_categoria: 'Eletrônicos' },
  { quantidade_produtos: 1, nome_categoria: 'Acessórios' }
]
```

Figura 22 – Resultado da Consulta 22 – Quantidade de produtos por categoria (O AUTOR, 2026)

Cada categoria — Eletrônicos, Informática e Acessórios — possui exatamente um produto cadastrado. Em um catálogo real, essa consulta identifica desequilíbrios na distribuição do mix de produtos entre categorias, informando decisões de expansão de catálogo.

Consulta 23 – Ranking de clientes por valor gasto com nome

Descrição: Combina os pedidos com a coleção de clientes para exibir o nome do cliente junto ao total gasto.

```
db.pedidos.aggregate([
  { $group: { _id: "$id_cliente", total_gasto: { $sum: "$valor_total" } } },
  {
    $lookup: {
      from: "clientes",
      localField: "_id",
      foreignField: "_id",
      as: "cliente"
    }
  },
  { $unwind: "$cliente" },
  { $project: { _id: 0, nome: "$cliente.nome", total_gasto: 1 } },
  { $sort: { total_gasto: -1 } }
])
```

Algoritmo 30 – Consulta 23 – Ranking de clientes por valor gasto com nome (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.pedidos.aggregate([{$group: { _id: "$id_cliente", total_gasto: { $sum: "$valor_total" } } }, {$lookup: { from: "clientes", localField: "_id", foreignField: "_id", as: "cliente" } }, {$unwind: "$cliente" }, {$project: { _id: 0, nome: "$cliente.nome", total_gasto: 1 } }, {$sort: { total_gasto: -1 } }])
[
  { total_gasto: Decimal128('8650.00'), nome: 'Carlos Silva' },
  { total_gasto: Decimal128('2500.00'), nome: 'Mariana Souza' },
  { total_gasto: Decimal128('300.00'), nome: 'Lucas Ferreira' }
]
```

Figura 23 – Resultado da Consulta 23 – Ranking de clientes por valor gasto com nome (O AUTOR, 2026)

Carlos Silva lidera o ranking com R\$ 4.650,00, seguido de Mariana Souza (R\$ 2.500,00) e Lucas Ferreira (R\$ 300,00). Esse ranking é o fundamento de programas de clientes VIP e estratégias de retenção diferenciadas por nível de engajamento financeiro.

Consulta 24 – Total e receita gerada por produto

Descrição: Usa \$unwind para expandir o array de itens de cada pedido e agrupa por produto para calcular quantidade vendida e receita gerada.

```
db.pedidos.aggregate([
  { $unwind: "$itens" },
  {
    $group: {
      _id: "$itens.nome_produto",
      total_vendido: { $sum: "$itens.quantidade" },
      receita_gerada: { $sum: "$itens.subtotal" }
    }
  },
  { $sort: { receita_gerada: -1 } }
])
```

Algoritmo 31 – Consulta 24 – Total e receita gerada por produto (O AUTOR, 2026)



```
Atlas atlas-k0pp3w-shard-0 [primary] ecommerce_nosql> db.pedidos.aggregate([{$unwind: "$itens"}, {$group: { _id: "$itens.nome_produto", total_vendido: { $sum: "$itens.quantidade" }, receita_gerada: { $sum: "$itens.subtotal" } } }, { $sort: { receita_gerada: -1 } }])
[
  {
    _id: 'Notebook Gamer',
    total_vendido: 1,
    receita_gerada: Decimal128('4500.00')
  },
  {
    _id: 'Smartphone',
    total_vendido: 1,
    receita_gerada: Decimal128('2500.00')
  },
  {
    _id: 'Mouse Gamer',
    total_vendido: 2,
    receita_gerada: Decimal128('300.00')
  }
]
```

Figura 24 – Resultado da Consulta 24 – Total e receita gerada por produto (O AUTOR, 2026)

O Notebook Gamer gerou R\$ 4.500,00 em receita com 1 unidade vendida; o Smartphone gerou R\$ 2.500,00 com 1 unidade; o Mouse Gamer gerou R\$ 300,00 com 2 unidades. Essa consulta é a base do relatório de produtos mais vendidos, essencial para gestão de catálogo e negociação com fornecedores.

Consulta 25 – Itens de pedido com preço acima de R\$ 1.000

Descrição: Expande o array de itens e filtra apenas os que possuem preço unitário acima de R\$ 1.000,00.

```
db.pedidos.aggregate([
  { $unwind: "$itens" },
  { $match: { "itens.preco_unitario": { $gt: NumberDecimal("1000.00") } } },
  {
    $project: {
      _id: 0,
      "itens.nome_produto": 1,
      "itens.quantidade": 1,
      "itens.preco_unitario": 1,
      "itens.subtotal": 1
    }
  }
])
```

Algoritmo 32 – Consulta 25 – Itens de pedido com preço acima de R\$ 1.000 (O AUTOR, 2026)



```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.pedidos.aggregate([{$unwind: "$itens"}, {$match: { "itens.preco_unitario": { $gt: NumberDecimal("1000.00") } } }, {$project: { _id: 0, "itens.nome_produto": 1, "itens.quantidade": 1, "itens.preco_unitario": 1, "itens.subtotal": 1 } }])
[
  {
    itens: {
      nome_produto: 'Notebook Gamer',
      quantidade: 1,
      preco_unitario: Decimal128('4500.00'),
      subtotal: Decimal128('4500.00')
    }
  },
  {
    itens: {
      nome_produto: 'Smartphone',
      quantidade: 1,
      preco_unitario: Decimal128('2500.00'),
      subtotal: Decimal128('2500.00')
    }
  }
]
```

Figura 25 – Resultado da Consulta 25 – Itens de pedido com preço acima de R\$ 1.000 (O AUTOR, 2026)

Notebook Gamer (R\$ 4.500,00) e Smartphone (R\$ 2.500,00) são retornados. Essa consulta pode ser usada para identificar itens de alto valor dentro dos pedidos, com aplicações em análise de risco e estratégias de parcelamento preferencial.

Categoria 4 — Consultas com \$lookup (Junções entre Coleções)

Consulta 26 – Pedidos com nome do cliente

Descrição: Realiza um \$lookup para trazer o nome do cliente junto aos dados de cada pedido.

```

db.pedidos.aggregate([
  {
    $lookup: {
      from: "clientes",
      localField: "id_cliente",
      foreignField: "_id",
      as: "cliente"
    }
  },
  { $unwind: "$cliente" },
  {
    $project: {
      _id: 0,
      "cliente.nome": 1,
      data_pedido: 1,
      valor_total: 1,
      status: 1
    }
  }
])

```

Algoritmo 33 – Consulta 26 – Pedidos com nome do cliente (O AUTOR, 2026)

```

Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.pedidos.aggregate([{$lookup: { from: "clientes", localField: "id_cliente", foreignField: "_id", as: "cliente" } }, {$unwind: "$cliente" }, {$project: { _id: 0, "cliente.nome": 1, data_pedido: 1, valor_total: 1, status: 1 } }])
[
  {
    data_pedido: ISODate('2026-03-10T00:00:00.000Z'),
    valor_total: Decimal128('4650.00'),
    status: 'Pago',
    cliente: { nome: 'Carlos Silva' }
  },
  {
    data_pedido: ISODate('2026-03-15T00:00:00.000Z'),
    valor_total: Decimal128('2500.00'),
    status: 'Pendente',
    cliente: { nome: 'Mariana Souza' }
  },
  {
    data_pedido: ISODate('2026-04-02T00:00:00.000Z'),
    valor_total: Decimal128('300.00'),
    status: 'Cancelado',
    cliente: { nome: 'Lucas Ferreira' }
  }
]

```

Figura 26 – Resultado da Consulta 26 – Pedidos com nome do cliente (O AUTOR, 2026)

Cada pedido é exibido com o nome do respectivo cliente, junto à data, valor e status. Essa consulta equivale ao INNER JOIN entre pedidos e clientes do modelo relacional, demonstrando como o \$lookup realiza junções no MongoDB.

Consulta 27 – Produtos com nome da categoria

Descrição: Enriquece cada produto com o nome de sua categoria por meio de \$lookup.

```

db.produtos.aggregate([
  {
    $lookup: {
      from: "categorias",

```

```

        localField: "id_categoria",
        foreignField: "_id",
        as: "categoria"
    }
},
{ $unwind: "$categoria" },
{
    $project: {
        _id: 0,
        nome: 1,
        preco: 1,
        "categoria.nome_categoria": 1
    }
}
])

```

Algoritmo 34 – Consulta 27 – Produtos com nome da categoria (O AUTOR, 2026)

```

Atlas atlas-k0pp3w-shard-0 [primary] ecommerce_nosql> db.produtos.aggregate([{$lookup: {from: "categorias", localField: "id_categoria", foreignField: "_id", as: "categoria" } }, {$unwind: "$categoria" }, {$project: { _id: 0, nome: 1, preco: 1, "categoria.nome_categoria": 1 } }])
[
  {
    nome: 'Notebook Gamer',
    preco: Decimal128('4500.00'),
    categoria: { nome_categoria: 'Informática' }
  },
  {
    nome: 'Mouse Gamer',
    preco: Decimal128('150.00'),
    categoria: { nome_categoria: 'Acessórios' }
  },
  {
    nome: 'Smartphone',
    preco: Decimal128('2500.00'),
    categoria: { nome_categoria: 'Eletrônicos' }
  }
]

```

Figura 27 – Resultado da Consulta 27 – Produtos com nome da categoria (O AUTOR, 2026)

Notebook Gamer aparece associado à categoria Informática, Mouse Gamer à categoria Acessórios e Smartphone à categoria Eletrônicos. Essa consulta é fundamental para exibição do catálogo de produtos com suas classificações tanto em interfaces de administração quanto na vitrine do cliente.

Consulta 28 – Produtos com nome do fornecedor

Descrição: Enriquece cada produto com o nome de seu fornecedor por meio de \$lookup.

```

db.produtos.aggregate([
  {
    $lookup: {
      from: "fornecedores",
      localField: "id_fornecedor",
      foreignField: "_id",
      as: "fornecedor"
    }
  },

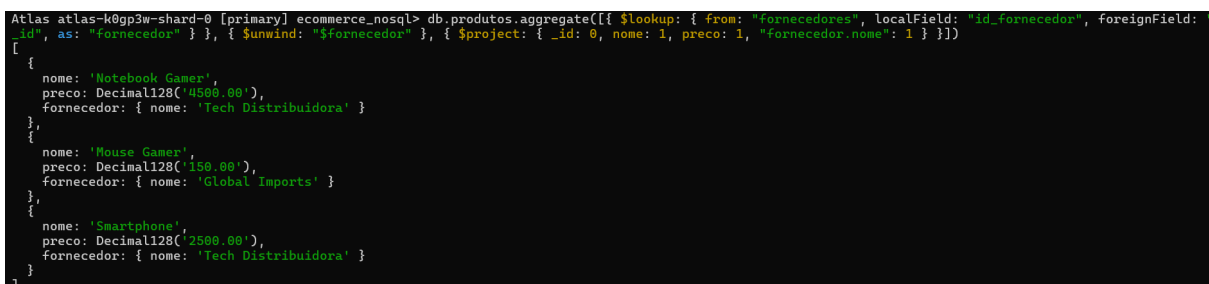
```

```

    { $unwind: "$fornecedor" },
    {
      $project: {
        _id: 0,
        nome: 1,
        preco: 1,
        "fornecedor.nome": 1
      }
    }
  ]
})

```

Algoritmo 35 – Consulta 28 – Produtos com nome do fornecedor (O AUTOR, 2026)



```

Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.produtos.aggregate([{$lookup: { from: "fornecedores", localField: "id_fornecedor", foreignField: "_id", as: "fornecedor" } }, {$unwind: "$fornecedor" }, {$project: { _id: 0, nome: 1, preco: 1, "fornecedor.nome": 1 } }])
[
  {
    nome: 'Notebook Gamer',
    preco: Decimal128('4500.00'),
    fornecedor: { nome: 'Tech Distribuidora' }
  },
  {
    nome: 'Mouse Gamer',
    preco: Decimal128('150.00'),
    fornecedor: { nome: 'Global Imports' }
  },
  {
    nome: 'Smartphone',
    preco: Decimal128('2500.00'),
    fornecedor: { nome: 'Tech Distribuidora' }
  }
]

```

Figura 28 – Resultado da Consulta 28 – Produtos com nome do fornecedor (O AUTOR, 2026)

Notebook Gamer e Smartphone são fornecidos pela Tech Distribuidora; Mouse Gamer é fornecido pela Global Imports. Essa visão é utilizada em processos de auditoria de fornecedores e análise de concentração de fornecimento, identificando riscos de dependência de um único parceiro.

Consulta 29 – Clientes sem pedidos registrados

Descrição: Identifica clientes que ainda não realizaram nenhum pedido no sistema.

```

db.clientes.aggregate([
  {
    $lookup: {
      from: "pedidos",
      localField: "_id",
      foreignField: "id_cliente",
      as: "pedidos"
    }
  },
  { $match: { pedidos: { $size: 0 } } },
  { $project: { _id: 0, nome: 1, email: 1 } }
])

```

Algoritmo 36 – Consulta 29 – Clientes sem pedidos registrados (O AUTOR, 2026)

```
Atlas atlas-k0gp3w-shard-0 [primary] ecommerce_nosql> db.clientes.aggregate([{$lookup: {from: "pedidos", localField: "_id", foreignField: "id_cliente", as: "pedidos" } }], {$match: {pedidos: { $size: 0 } } }], {$project: { _id: 0, nome: 1, email: 1 } }])
```

Figura 29 – Resultado da Consulta 29 – Clientes sem pedidos registrados (O AUTOR, 2026)

Com os dados atuais, nenhum cliente sem pedidos é encontrado, pois todos os três clientes possuem ao menos um pedido registrado. Em um sistema real com base maior de clientes, essa consulta é essencial para campanhas de reativação — identificando usuários cadastrados que nunca converteram.

Consulta 30 – Visão consolidada dos pedidos

Descrição: Monta uma visão completa de cada pedido, trazendo o nome do cliente por \$lookup e apresentando os arrays de itens e pagamentos embutidos.

```
db.pedidos.aggregate([
  {
    $lookup: {
      from: "clientes",
      localField: "id_cliente",
      foreignField: "_id",
      as: "cliente"
    }
  },
  { $unwind: "$cliente" },
  {
    $project: {
      _id: 0,
      "cliente.nome": 1,
      "cliente.email": 1,
      data_pedido: 1,
      valor_total: 1,
      status: 1,
      itens: 1,
      pagamentos: 1
    }
  }
])
```

Algoritmo 37 – Consulta 30 – Visão consolidada dos pedidos (O AUTOR, 2026)

```

Atlas atlas-h0gn3w-shard-0 [primary] ecommerce_nosql> db.pedidos.aggregate([{$lookup: { from: "clientes", localField: "id_cliente", foreignField: "_id", as: "cliente" } }, { $unwind: "$cliente" }, { $project
: { _id: 0, "cliente_nome": 1, "cliente_email": 1, data_pedido: 1, valor_total: 1, status: 1, itens: 1, pagamentos: 1 } }])
[
  {
    data_pedido: ISODate('2026-03-10T00:00:00.000Z'),
    valor_total: Decimal128('4650.00'),
    status: 'Pago',
    itens: [
      {
        id_produto: ObjectId('6a36baba37377719f3abc11c'),
        nome_produto: 'Notebook Gamer',
        quantidade: 1,
        preco_unitario: Decimal128('4500.00'),
        subtotal: Decimal128('4500.00')
      },
      {
        id_produto: ObjectId('6a36baba37377719f3abc11d'),
        nome_produto: 'Mouse Gamer',
        quantidade: 1,
        preco_unitario: Decimal128('150.00'),
        subtotal: Decimal128('150.00')
      }
    ],
    pagamentos: [
      {
        tipo_pagamento: 'Cartão de Crédito',
        status_pagamento: 'Aprovado',
        data_pagamento: ISODate('2026-03-10T00:00:00.000Z'),
        valor_pago: Decimal128('4650.00')
      }
    ],
    cliente: { nome: 'Carlos Silva', email: 'carlos@gmail.com' }
  },
  {
    data_pedido: ISODate('2026-03-15T00:00:00.000Z'),
    valor_total: Decimal128('2500.00'),
    status: 'Pendente',
    itens: [
      {
        id_produto: ObjectId('6a36baba37377719f3abc11e'),
        nome_produto: 'Smartphone',
        quantidade: 1,
        preco_unitario: Decimal128('2500.00'),
        subtotal: Decimal128('2500.00')
      }
    ],
    pagamentos: [
      {
        tipo_pagamento: 'PIX',
        status_pagamento: 'Pendente',
        data_pagamento: ISODate('2026-03-15T00:00:00.000Z'),
        valor_pago: Decimal128('2500.00')
      }
    ],
    cliente: { nome: 'Mariana Souza', email: 'mariana@gmail.com' }
  },
  {
    data_pedido: ISODate('2026-04-02T00:00:00.000Z'),
    valor_total: Decimal128('300.00'),
    status: 'Cancelado',
    itens: [
      {
        id_produto: ObjectId('6a36baba37377719f3abc11f'),
        nome_produto: 'Mouse Gamer',
        quantidade: 2,
        preco_unitario: Decimal128('150.00'),
        subtotal: Decimal128('300.00')
      }
    ],
    pagamentos: [
      {
        tipo_pagamento: 'Boleto',
        status_pagamento: 'Cancelado',
        data_pagamento: ISODate('2026-04-02T00:00:00.000Z'),
        valor_pago: Decimal128('0.00')
      }
    ],
    cliente: { nome: 'Lucas Ferreira', email: 'lucas@gmail.com' }
  }
]

```

Figura 30 – Resultado da Consulta 30 – Visão consolidada dos pedidos (O AUTOR, 2026)

Cada documento retornado apresenta o nome e e-mail do cliente, a data e valor do pedido, seu status, todos os itens comprados com seus subtotais e o registro completo de pagamento — tudo em uma única consulta. Essa é a consulta que mais evidencia a vantagem do modelo de documentos com embedding: o que no SQL demandaria junções entre quatro tabelas é resolvido aqui com um único \$lookup, pois itens e pagamentos já habitam o documento de pedido.

5 CONCLUSÃO

O desenvolvimento deste projeto possibilitou a aplicação prática dos conceitos de banco de dados não relacional ao mesmo domínio de e-commerce trabalhado anteriormente no paradigma relacional. Essa duplicidade de implementação revelou-se especialmente rica do ponto de vista de aprendizagem, pois tornou explícitas as diferenças fundamentais entre os dois paradigmas — não apenas do ponto de vista técnico, mas também no nível das decisões de modelagem e das filosofias que as sustentam.

No modelo relacional, a normalização é o princípio guia: os dados são decompostos em tabelas independentes para eliminar redundância, e as consultas reconstróem as entidades compostas por meio de JOINS. No modelo de documentos do MongoDB, o princípio guia é o padrão de acesso: os dados são organizados de forma que as operações mais frequentes exijam o mínimo de leituras e nenhum JOIN. Isso implica aceitar algum grau de redundância — como o campo `nome_produto` armazenado dentro de cada item de pedido — em troca de desempenho e simplicidade nas consultas mais comuns.

As trinta consultas desenvolvidas demonstraram a expressividade da MongoDB Query Language em diferentes níveis de complexidade. As consultas simples evidenciaram a intuitividade dos operadores de filtragem e projeção. As consultas de agregação mostraram a flexibilidade do framework de pipeline para análises analíticas sofisticadas. As consultas com `$lookup` demonstraram que, mesmo no modelo de documentos, é possível realizar junções entre coleções quando o embedding não é adequado — como nos relacionamentos entre produtos e suas categorias ou fornecedores.

Os resultados obtidos confirmaram que o MongoDB é uma escolha técnica sólida para sistemas de e-commerce, em especial para os módulos de catálogo de produtos e gestão de pedidos, onde a variabilidade de atributos e a frequência de leituras completas de entidades compostas tornam o modelo de documentos mais eficiente que o relacional.

Como sugestões para melhorias futuras, destacam-se: a implementação de índices compostos nas coleções de pedidos e produtos para otimização das consultas de maior volume; a adoção de validação de esquema com JSON Schema no nível da coleção para garantir integridade dos dados inseridos; a exploração do

recurso de transações multi-documento do MongoDB para garantir atomicidade nas operações de criação de pedido que envolvem atualização simultânea de estoque; e a integração com o MongoDB Atlas para explorar recursos de busca textual, análise de dados em tempo real e escalabilidade em nuvem.

Em suma, o desenvolvimento do presente trabalho contribuiu de modo significativo para a consolidação dos conhecimentos relacionados à modelagem e implementação de bancos de dados não relacionais, aproximando os conceitos acadêmicos das situações reais encontradas no mercado de tecnologia.

REFERÊNCIAS

BANKER, Kyle. **MongoDB in Action**. 2. ed. Shelter Island: Manning Publications, 2016.

CHODOROW, Kristina. **MongoDB: The Definitive Guide**. 3. ed. Sebastopol: O'Reilly Media, 2019.

DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. Rio de Janeiro: Elsevier, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant. **Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson, 2019.

HEWITT, Eben. **Cassandra: The Definitive Guide**. 3. ed. Sebastopol: O'Reilly Media, 2020.

MONGODB INC. **MongoDB Documentation**. Disponível em: <https://www.mongodb.com/docs/>. Acesso em: 19 jun. 2026.

MONGODB INC. **Aggregation Pipeline**. Disponível em: <https://www.mongodb.com/docs/manual/core/aggregation-pipeline/>. Acesso em: 19 jun. 2026.

SADALAGE, Pramod J.; FOWLER, Martin. **NoSQL Essencial: Um Guia Conciso para o Mundo Emergente da Persistência Poliglota**. São Paulo: Novatec, 2013.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 6. ed. Rio de Janeiro: Elsevier, 2012.